

# **Competitive Programming**

## **Section 37: Segment Trees**

Competitive Programming Course

2026-08-28

# Table of contents

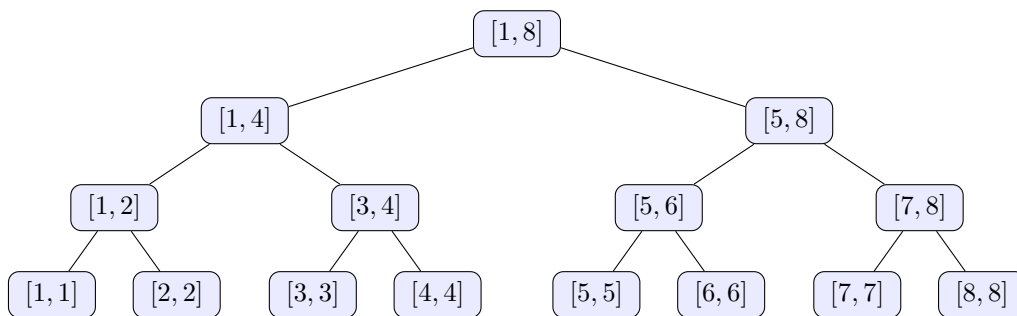
|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| <b>1</b>  | <b>Segment trees</b>                                      | <b>3</b>  |
| 1.1       | Build intervals by repeatedly splitting in half . . . . . | 3         |
| <b>2</b>  | <b>Point update and range query</b>                       | <b>4</b>  |
| 2.1       | Point update: repair one path . . . . .                   | 4         |
| 2.2       | Range query: take whole nodes whenever possible . . . . . | 4         |
| <b>3</b>  | <b>Rich node states</b>                                   | <b>6</b>  |
| 3.1       | Start from the merge question . . . . .                   | 6         |
| 3.2       | Derive each merge formula . . . . .                       | 6         |
| 3.3       | Worked merge . . . . .                                    | 7         |
| <b>4</b>  | <b>Lazy propagation</b>                                   | <b>8</b>  |
| 4.1       | Why ordinary recursion becomes too slow . . . . .         | 8         |
| 4.2       | What a lazy tag means . . . . .                           | 8         |
| 4.3       | When to push . . . . .                                    | 9         |
| <b>5</b>  | <b>Complete implementation templates</b>                  | <b>10</b> |
| 5.1       | Point assignment and range minimum: C++ . . . . .         | 10        |
| 5.2       | Point assignment and range minimum: Python . . . . .      | 12        |
| 5.3       | Iterative point assignment and range minimum . . . . .    | 14        |
| 5.4       | Range addition and range sum: C++ . . . . .               | 17        |
| 5.5       | Range addition and range sum: Python . . . . .            | 19        |
| <b>6</b>  | <b>Mistakes to avoid</b>                                  | <b>23</b> |
| <b>7</b>  | <b>Local exercises</b>                                    | <b>24</b> |
| <b>8</b>  | <b>How to use the checker</b>                             | <b>25</b> |
| <b>9</b>  | <b>Choosing and proving a segment tree</b>                | <b>26</b> |
| 9.1       | Trace: iterative range minimum . . . . .                  | 26        |
| 9.2       | Trace: maximum subarray merge . . . . .                   | 26        |
| 9.3       | Lazy trace . . . . .                                      | 26        |
| <b>10</b> | <b>Implementation choices and failure modes</b>           | <b>27</b> |
| 10.1      | Formative checks . . . . .                                | 27        |
| <b>11</b> | <b>Submission workflow</b>                                | <b>28</b> |

# 1 Segment trees

A segment tree is useful when we repeatedly ask about an interval, but updates make a static prefix table obsolete. It stores summaries for a hierarchy of intervals so that both operations touch only a small part of that hierarchy.

## 1.1 Build intervals by repeatedly splitting in half

The root owns the whole array. Split its interval into a left half and a right half, then repeat until every leaf owns one array position:



These are not eight independent paths. Every parent interval is exactly the union of its two children, so its summary can be computed by merging the two child summaries.

For range sums:

```
tree[node] = tree[left child] + tree[right child]
```

For range minimum:

```
tree[node] = min(tree[left child], tree[right child])
```

The precise invariant is:

```
tree[node] stores the correct summary
of exactly the node's interval.
```

Build the leaves from the array, then compute internal nodes bottom-up. The tree has  $n$  leaves and fewer than  $n$  branching nodes when  $n$  is a power of two, so construction and memory are both  $O(n)$ .

The merge must be associative: grouping adjacent pieces in a different way must not change the answer. This works for `sum`, `min`, `max`, `gcd`, and many custom states. The identity value used for an empty piece depends on the merge: `0` for sums, `+infinity` for minimum, and so on.

## 2 Point update and range query

### 2.1 Point update: repair one path

Suppose nodes store minimums for this 1-based array:

```
a = [7, 2, 6, 3]
```

Leaves store:

```
[1,1]=7  [2,2]=2  [3,3]=6  [4,4]=3
```

Internal minimum nodes:

```
[1,2]=min(7,2)=2  
[3,4]=min(6,3)=3  
[1,4]=min(2,3)=2
```

If  $a[2]$  becomes 8, first change leaf  $[2,2]$ . Every interval not containing position 2 still summarizes unchanged values, so only its ancestors can be wrong:

```
[2,2] = 8  
  |  
  v  
[1,2] = min(7,8) = 7  
  |  
  v  
[1,4] = min(7,3) = 3
```

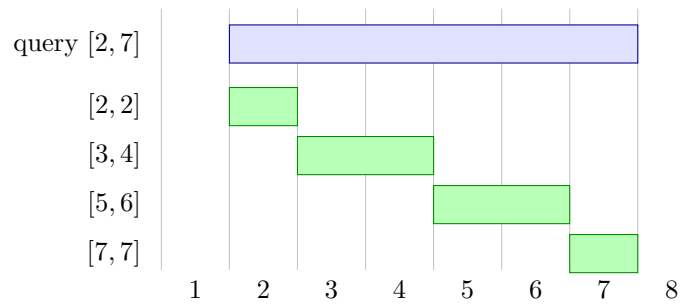
The interval length doubles at every step toward the root. There are only  $O(\log n)$  levels, so the assignment and all necessary repairs take  $O(\log n)$  time.

### 2.2 Range query: take whole nodes whenever possible

For a query  $[ql,qr]$ , compare it with the interval of the current node:

```
no overlap      -> ignore this node and return the identity  
fully contained -> use this node's stored summary  
                  and stop descending  
partial overlap -> query both children and merge their answers
```

For example, querying  $[2,7]$  in a tree over eight positions selects four stored nodes:



Their intervals are disjoint and their union is exactly  $[2, 7]$ :

$$[2, 7] = [2, 2] + [3, 4] + [5, 6] + [7, 7]$$

Here is the recursion depth by depth. The root  $[1, 8]$  is only partially inside  $[2, 7]$ , so inspect its children. At the next depth:

$[1, 4]$  is partial because it contains the left boundary 2  
 $[5, 8]$  is partial because it contains the right boundary 7

Inspecting their children gives:

$[1, 2]$  is partial  
 $[3, 4]$  is fully inside  
 $[5, 6]$  is fully inside  
 $[7, 8]$  is partial

For  $[3, 4]$ , use its stored summary immediately. Do not descend to  $[3, 3]$  and  $[4, 4]$ ; this is what it means to take a whole node. Likewise, use  $[5, 6]$  without opening it.

Only the two partial nodes need another step:

$[1, 1]$  is outside       $[2, 2]$  is fully inside  
 $[7, 7]$  is fully inside     $[8, 8]$  is outside

At any depth, at most one partial node contains the left query boundary and at most one contains the right query boundary. Nodes strictly between them are fully inside and are used without further descent; nodes outside are ignored. Thus only a constant number of nodes needs inspection per depth. The tree has  $O(\log n)$  depths, so the complete query takes  $O(\log n)$  time.

## 3 Rich node states

### 3.1 Start from the merge question

A node does not have to store one number. It should store exactly enough information to merge two adjacent segments without reopening their elements.

For maximum subarray sum, suppose the two children contain:

```
left child L = [-5, 4]    best subarray inside L is [4], sum 4
right child R = [ 3,-5]    best subarray inside R is [3], sum 3
```

If the parent stored only  $\max(L.\text{best}, R.\text{best})$ , it would answer 4. But after joining the children, the parent array is  $[-5, 4, 3, -5]$ , whose best subarray is  $[4, 3]$  with sum 7. It starts in the left child and finishes in the right child, so neither child's answer contains it completely.

Any subarray that crosses this boundary has a forced shape:

```
a suffix of the left child + a prefix of the right child
```

This is why each node stores four quantities:

```
sum   = total sum of the segment
pref  = best non-empty prefix sum
suff  = best non-empty suffix sum
best  = best non-empty subarray sum
```

`pref` needs `sum` because a parent prefix may contain the entire left child and continue into the right child. Symmetrically, `suff` needs `sum` to cross from the right child into the left.

### 3.2 Derive each merge formula

Let `L` and `R` be adjacent child states.

The parent total has only one possibility:

```
sum = L.sum + R.sum
```

A parent prefix either stops inside `L`, or takes all of `L` and continues with the best prefix of `R`:

```
pref = max(L.pref, L.sum + R.pref)
```

A parent suffix is symmetric:

```
suff = max(R.suff, R.sum + L.suff)
```

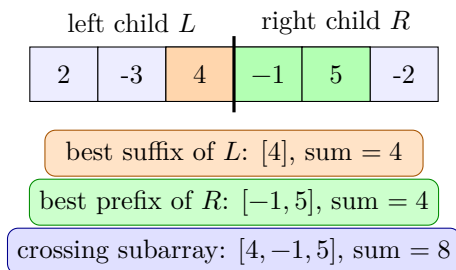
Finally, the best subarray lies entirely in L, entirely in R, or crosses the boundary:

```
best = max(L.best, R.best, L.suff + R.pref)
```

There is no fourth case: every subarray either crosses the boundary or it does not.

### 3.3 Worked merge

Merge  $L = [2, -3, 4]$  and  $R = [-1, 5, -2]$ . The highlighted crossing candidate uses the best suffix of L and the best prefix of R:



The child states are:

```
L: sum=3, pref=3, suff=4, best=4  
R: sum=2, pref=4, suff=3, best=5
```

The merged state is:

```
sum  = 3 + 2          = 5  
pref = max(3, 3 + 4)  = 7  
suff = max(3, 2 + 4)  = 6  
best = max(4, 5, 4 + 4) = 8
```

This is the general design method for rich segment-tree states: list every possible shape of an answer after joining two halves, then store the boundary information needed to evaluate those shapes.

## 4 Lazy propagation

### 4.1 Why ordinary recursion becomes too slow

A point update follows one root-to-leaf path. A range update may cover almost every leaf; changing them separately would cost  $O(n)$ . But if an update fully covers a node interval, we already know its effect on the node summary without visiting any child.

For a range-add/range-sum tree, adding `delta` to a node of length `len` increases its sum by `delta * len`:

```
seg[node] += delta * len
lazy[node] += delta
```

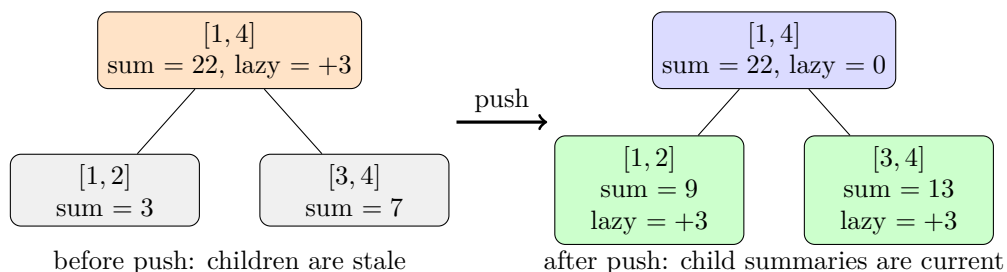
The first line makes the node's own summary correct immediately. The lazy tag records work that its children have not received yet. Then the update stops at this node.

### 4.2 What a lazy tag means

The invariant is deliberately asymmetric:

```
seg[node] already includes every update
affecting the whole node interval.
lazy[node] is the addition still waiting
to be applied to both children.
```

Suppose `[1,4]` originally has sum 10, with child sums 3 and 7. After adding `+3` to all four positions, its new sum is  $10 + 3 \cdot 4 = 22$ . Before a push, only the parent has been updated. After a push, both children receive the tag and the parent tag is cleared:



The total remains  $9 + 13 = 22$ . Pushing does not apply the update twice; it merely transfers the deferred information one level downward.

## 4.3 When to push

If a query or update fully contains the current node, use or update that node and stop. If it only partially overlaps, its children must be inspected. Push the pending tag first so their summaries describe the correct values, recurse, then merge the children back into the parent.

```
range operation at node:
  no overlap      -> return
  full cover      -> apply to this node and stop
  partial overlap -> push, recurse, then merge children
```

Range addition tags compose by addition: two deferred updates  $+3$  and  $-1$  become one tag  $+2$ . Other update types need different composition rules. For example, a later assignment replaces an earlier assignment rather than adding to it.

A range operation uses the same interval decomposition as the earlier query. For example,  $[2,7]$  is represented by the fully covered nodes:

```
[2,2], [3,4], [5,6], [7,7]
```

For an update, apply the lazy tag once to each of these nodes and stop there; do not visit their leaves. At every tree depth, only the node containing the left boundary and the node containing the right boundary can remain partially covered and require further descent. Each of those two paths has  $O(\log n)$  levels, and only a constant amount of work is done at each visited node. Therefore both a range update and a range query take  $O(\log n)$  time.

## 5 Complete implementation templates

### 5.1 Point assignment and range minimum: C++

This version follows the recursive cases from Chapter 2. Public methods use 1-based inclusive indices.

```
struct SegmentTreeMin {
    static constexpr long long INF = (1LL << 60);

    int n;
    vector<long long> tree;

    explicit SegmentTreeMin(const vector<long long>& values)
        : n(static_cast<int>(values.size())), tree(4 * n, INF) {
        build(1, 1, n, values);
    }

    void build(
        int node,
        int left,
        int right,
        const vector<long long>& values
    ) {
        if (left == right) {
            tree[node] = values[left - 1];
            return;
        }
        int middle = (left + right) / 2;
        build(2 * node, left, middle, values);
        build(2 * node + 1, middle + 1, right, values);
        pull(node);
    }

    void pull(int node) {
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }

    void assign(int position, long long value) {
        assign(1, 1, n, position, value);
    }

    void assign(
        int node,
        int left,
```

```

    int right,
    int position,
    long long value
) {
    if (left == right) {
        tree[node] = value;
        return;
    }
    int middle = (left + right) / 2;
    if (position <= middle) {
        assign(2 * node, left, middle, position, value);
    } else {
        assign(
            2 * node + 1,
            middle + 1,
            right,
            position,
            value
        );
    }
    pull(node);
}

long long query(int query_left, int query_right) const {
    return query(1, 1, n, query_left, query_right);
}

long long query(
    int node,
    int left,
    int right,
    int query_left,
    int query_right
) const {
    if (query_left <= left && right <= query_right) {
        return tree[node];
    }
    int middle = (left + right) / 2;
    long long answer = INF;
    if (query_left <= middle) {
        answer = min(
            answer,
            query(
                2 * node,
                left,
                middle,
                query_left,
                query_right
            )
        );
    }
}

```

```

        if (middle < query_right) {
            answer = min(
                answer,
                query(
                    2 * node + 1,
                    middle + 1,
                    right,
                    query_left,
                    query_right
                )
            );
        }
        return answer;
    }
};

```

## 5.2 Point assignment and range minimum: Python

```

class SegmentTreeMin:
    INF = 10**30

    def __init__(self, values):
        self.n = len(values)
        self.tree = [self.INF] * (4 * self.n)
        self._build(1, 1, self.n, values)

    def _build(self, node, left, right, values):
        if left == right:
            self.tree[node] = values[left - 1]
            return
        middle = (left + right) // 2
        self._build(2 * node, left, middle, values)
        self._build(2 * node + 1, middle + 1, right, values)
        self._pull(node)

    def _pull(self, node):
        self.tree[node] = min(
            self.tree[2 * node],
            self.tree[2 * node + 1],
        )

    def assign(self, position, value):
        self._assign(1, 1, self.n, position, value)

    def _assign(self, node, left, right, position, value):
        if left == right:
            self.tree[node] = value
            return

```

```

middle = (left + right) // 2
if position <= middle:
    self._assign(2 * node, left, middle, position, value)
else:
    self._assign(
        2 * node + 1,
        middle + 1,
        right,
        position,
        value,
    )
self._pull(node)

def query(self, query_left, query_right):
    return self._query(
        1,
        1,
        self.n,
        query_left,
        query_right,
    )

def _query(
    self,
    node,
    left,
    right,
    query_left,
    query_right,
):
    if query_left <= left and right <= query_right:
        return self.tree[node]
    middle = (left + right) // 2
    answer = self.INF
    if query_left <= middle:
        answer = min(
            answer,
            self._query(
                2 * node,
                left,
                middle,
                query_left,
                query_right,
            ),
        )
    if middle < query_right:
        answer = min(
            answer,
            self._query(
                2 * node + 1,
                middle + 1,

```

```

        right,
        query_left,
        query_right,
    ),
)
return answer

```

## 5.3 Iterative point assignment and range minimum

The recursive implementation mirrors the interval tree and is easier to derive. The iterative version stores the same tree in one flat array:

```

tree[1]          root
tree[2], tree[3] children of the root
tree[size..2*size-1] leaves

```

`size` is the first power of two at least `n`. Extra leaves are filled with the minimum identity `INF`. A query changes its 1-based inclusive range `[l,r]` into a half-open range of leaf indices `[l,r+1)`. While moving both boundaries upward, an odd left boundary and an odd right boundary expose nodes that belong entirely to the answer.

### 5.3.1 C++

```

struct IterativeSegmentTreeMin {
    static constexpr long long INF = (1LL << 60);

    int size = 1;
    vector<long long> tree;

    explicit IterativeSegmentTreeMin(
        const vector<long long>& values
    ) {
        while (size < static_cast<int>(values.size())) {
            size *= 2;
        }

        // Leaves occupy tree[size], tree[size+1], ... .
        tree.assign(2 * size, INF);
        for (int index = 0; index < int(values.size()); ++index) {
            tree[size + index] = values[index];
        }

        // Build every parent from its two children.
        for (int node = size - 1; node >= 1; --node) {
            tree[node] = min(
                tree[2 * node],
                tree[2 * node + 1]
            );
        }
    }
};

```

```

    );
}
}

void assign(int position, long long value) {
    // Convert a 1-based array position to its leaf.
    int node = size + position - 1;
    tree[node] = value;

    // Repair ancestors until reaching the root.
    for (node /= 2; node >= 1; node /= 2) {
        tree[node] = min(
            tree[2 * node],
            tree[2 * node + 1]
        );
    }
}

long long query(int left, int right) const {
    // Convert inclusive [left,right] to half-open leaf range.
    left = size + left - 1;
    right = size + right;
    long long answer = INF;

    while (left < right) {
        if (left % 2 == 1) {
            // left is a right child fully inside the range.
            answer = min(answer, tree[left]);
            ++left;
        }
        if (right % 2 == 1) {
            // The node just before right is fully inside.
            --right;
            answer = min(answer, tree[right]);
        }

        // Move the remaining half-open range to its parents.
        left /= 2;
        right /= 2;
    }
    return answer;
}
};

```

The assignment loop ends safely: after repairing `tree[1]`, integer division changes the node to zero and the condition fails.

### 5.3.2 Python

```
class IterativeSegmentTreeMin:
    INF = 10**30

    def __init__(self, values):
        self.size = 1
        while self.size < len(values):
            self.size *= 2

        # Leaves begin at index self.size.
        self.tree = [self.INF] * (2 * self.size)
        for index, value in enumerate(values):
            self.tree[self.size + index] = value

        # Build parents from bottom to top.
        for node in range(self.size - 1, 0, -1):
            self.tree[node] = min(
                self.tree[2 * node],
                self.tree[2 * node + 1],
            )

    def assign(self, position, value):
        # Convert the 1-based position to a leaf.
        node = self.size + position - 1
        self.tree[node] = value

        # Repair every ancestor.
        node //= 2
        while node >= 1:
            self.tree[node] = min(
                self.tree[2 * node],
                self.tree[2 * node + 1],
            )
            node //= 2

    def query(self, left, right):
        # Convert inclusive [left,right] to half-open leaf range.
        left = self.size + left - 1
        right = self.size + right
        answer = self.INF

        while left < right:
            if left % 2 == 1:
                # This right-child node is fully inside.
                answer = min(answer, self.tree[left])
                left += 1
            if right % 2 == 1:
                # The node immediately before right is inside.
                right -= 1
```

```

        answer = min(answer, self.tree[right])

        # Move the unprocessed range to the parent level.
        left //= 2
        right //= 2

    return answer

```

## 5.4 Range addition and range sum: C++

Here `tree[node]` is always current. `lazy[node]` stores the addition still waiting for the children.

```

struct LazySegmentTree {
    int n;
    vector<long long> tree;
    vector<long long> lazy;

    explicit LazySegmentTree(const vector<long long>& values)
        : n(static_cast<int>(values.size())),
          tree(4 * n, 0),
          lazy(4 * n, 0) {
        build(1, 1, n, values);
    }

    void build(
        int node,
        int left,
        int right,
        const vector<long long>& values
    ) {
        if (left == right) {
            tree[node] = values[left - 1];
            return;
        }
        int middle = (left + right) / 2;
        build(2 * node, left, middle, values);
        build(2 * node + 1, middle + 1, right, values);
        pull(node);
    }

    void pull(int node) {
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    void apply(int node, int length, long long delta) {
        tree[node] += delta * length;
        lazy[node] += delta;
    }
}

```

```

void push(int node, int left, int right) {
    if (lazy[node] == 0 || left == right) return;
    int middle = (left + right) / 2;
    apply(2 * node, middle - left + 1, lazy[node]);
    apply(2 * node + 1, right - middle, lazy[node]);
    lazy[node] = 0;
}

void add(int query_left, int query_right, long long delta) {
    add(1, 1, n, query_left, query_right, delta);
}

void add(
    int node,
    int left,
    int right,
    int query_left,
    int query_right,
    long long delta
) {
    if (query_left <= left && right <= query_right) {
        apply(node, right - left + 1, delta);
        return;
    }
    push(node, left, right);
    int middle = (left + right) / 2;
    if (query_left <= middle) {
        add(
            2 * node,
            left,
            middle,
            query_left,
            query_right,
            delta
        );
    }
    if (middle < query_right) {
        add(
            2 * node + 1,
            middle + 1,
            right,
            query_left,
            query_right,
            delta
        );
    }
    pull(node);
}

long long query(int query_left, int query_right) {

```

```

        return query(1, 1, n, query_left, query_right);
    }

    long long query(
        int node,
        int left,
        int right,
        int query_left,
        int query_right
    ) {
        if (query_left <= left && right <= query_right) {
            return tree[node];
        }
        push(node, left, right);
        int middle = (left + right) / 2;
        long long answer = 0;
        if (query_left <= middle) {
            answer += query(
                2 * node,
                left,
                middle,
                query_left,
                query_right
            );
        }
        if (middle < query_right) {
            answer += query(
                2 * node + 1,
                middle + 1,
                right,
                query_left,
                query_right
            );
        }
        return answer;
    }
};

```

## 5.5 Range addition and range sum: Python

```

class LazySegmentTree:
    def __init__(self, values):
        self.n = len(values)
        self.tree = [0] * (4 * self.n)
        self.lazy = [0] * (4 * self.n)
        self._build(1, 1, self.n, values)

    def _build(self, node, left, right, values):

```

```

        if left == right:
            self.tree[node] = values[left - 1]
            return
        middle = (left + right) // 2
        self._build(2 * node, left, middle, values)
        self._build(2 * node + 1, middle + 1, right, values)
        self._pull(node)

    def _pull(self, node):
        self.tree[node] = (
            self.tree[2 * node] + self.tree[2 * node + 1]
        )

    def _apply(self, node, length, delta):
        self.tree[node] += delta * length
        self.lazy[node] += delta

    def _push(self, node, left, right):
        if self.lazy[node] == 0 or left == right:
            return
        middle = (left + right) // 2
        self._apply(
            2 * node,
            middle - left + 1,
            self.lazy[node],
        )
        self._apply(
            2 * node + 1,
            right - middle,
            self.lazy[node],
        )
        self.lazy[node] = 0

    def add(self, query_left, query_right, delta):
        self._add(
            1,
            1,
            self.n,
            query_left,
            query_right,
            delta,
        )

    def _add(
        self,
        node,
        left,
        right,
        query_left,
        query_right,
        delta,

```

```

):
    if query_left <= left and right <= query_right:
        self._apply(node, right - left + 1, delta)
        return
    self._push(node, left, right)
    middle = (left + right) // 2
    if query_left <= middle:
        self._add(
            2 * node,
            left,
            middle,
            query_left,
            query_right,
            delta,
        )
    if middle < query_right:
        self._add(
            2 * node + 1,
            middle + 1,
            right,
            query_left,
            query_right,
            delta,
        )
    self._pull(node)

def query(self, query_left, query_right):
    return self._query(
        1,
        1,
        self.n,
        query_left,
        query_right,
    )

def _query(
    self,
    node,
    left,
    right,
    query_left,
    query_right,
):
    if query_left <= left and right <= query_right:
        return self.tree[node]
    self._push(node, left, right)
    middle = (left + right) // 2
    answer = 0
    if query_left <= middle:
        answer += self._query(
            2 * node,

```

```
        left,  
        middle,  
        query_left,  
        query_right,  
    )  
    if middle < query_right:  
        answer += self._query(  
            2 * node + 1,  
            middle + 1,  
            right,  
            query_left,  
            query_right,  
        )  
    return answer
```

## 6 Mistakes to avoid

Mixing inclusive and half-open ranges.

Forgetting that maximum subarray is non-empty in this section.

Applying lazy delta without multiplying by segment length.

Descending into a partial node without pushing its lazy tag.

## 7 Local exercises

- A. Range Minimum Updates: Point-assign values and answer range minimum queries.
- B. Maximum Subarray Updates: After each point assignment, print the maximum non-empty subarray sum of the whole array.
- C. Lazy Range Add Sum: Support range add updates and range sum queries.

## 8 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/37_segment_trees/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.  
# Exercises

Use A to define a range-minimum merge, B to include prefix/suffix/best values in a merge, and C to state a lazy tag before coding. Test an update on one element, a full-range update, and tag composition order.

## 9 Choosing and proving a segment tree

Use a segment tree when updates invalidate precomputed answers and the requested range summary can be merged from two adjacent summaries. The node invariant is:

```
node v stores the exact summary of its own interval.
```

For a point assignment, only the leaf and its ancestors change. For a range query, the iterative or recursive walk selects disjoint canonical intervals whose union is the query range. Associativity of the merge makes their combined summary equal the answer.

### 9.1 Trace: iterative range minimum

For  $a=[5,4,3,2,1]$ , change position 3 to 10. The leaf becomes 10; recomputing its parent, grandparent, and root restores the invariant. Query  $[2,4]$  sees the values  $[4,10,2]$ , so its minimum is 2. In the half-open iterative form, translate 1-based inclusive  $[l,r]$  to leaf pointers  $[l-1+size, r+size)$ . The right endpoint is deliberately exclusive.

### 9.2 Trace: maximum subarray merge

Merge left  $[1,-2]$  and right  $[3,4]$ . Left state is  $sum=-1, pref=1, suff=-1, best=1$ ; right state is  $sum=7, pref=7, suff=7, best=7$ . The crossing candidate is  $-1+7=6$ , so the best is 7. The crossing candidate is always  $left.suff+right.pref$ ; no other crossing subarray is possible.

### 9.3 Lazy trace

If a node for  $[1,4]$  has sum 10 and receives  $+3$ , its summary becomes 22 and its pending tag becomes 3. Do not descend. If a later request partially touches the node, push: both length-two children gain  $3*2$  in their sums and gain a pending  $+3$ . Pushing preserves the parent summary while making child summaries truthful enough to recurse.

## 10 Implementation choices and failure modes

Pad the tree to a power of two. For unused leaves of a minimum tree use `INF`; for non-empty maximum-subarray trees use neutral (`sum=0, pref=suff=best=-INF`), never all zeros. For lazy range-add/range-sum, `apply(v, len, x)` must change the sum by `x*len`, and tags compose by addition. Use `long long` in C++ and ordinary Python integers for all sums.

| Mistake                                       | Revealing test                   |
|-----------------------------------------------|----------------------------------|
| use zero as min padding                       | all array values positive        |
| allow empty subarray                          | all values negative              |
| forget length in lazy apply                   | full-range update then sum query |
| descend without push                          | full update, then partial query  |
| mix <code>[l,r]</code> and <code>[l,r)</code> | one-element query                |

### 10.1 Formative checks

1. Why does `best` need `suff` and `pref`? Answer: a best subarray crossing a merge boundary consists exactly of the left suffix plus the right prefix.
2. What does a lazy tag mean? Answer: every element in the node's interval has received that unapplied delta, while `seg[v]` already includes its effect.
3. What is the answer after assigning `-5` to every element of `[-1,-2]`? Answer: `-5`; the section requires a non-empty subarray.

## 11 Submission workflow

Stress a tree against a literal array for short random sequences. Include overlapping lazy updates, negative deltas, assignment of the same position many times, and queries at both boundaries. Run:

```
CP_TARGET=solution python3 sections/37_segment_trees/check.py
```